

Database Design (Stage 3) — NBA Analytics

1) Database Implementation

Database name: nba_analytics

Database Connection verified through Google Cloud Shell:

```
alexisalvarez870@cloudshell:~ (cs411-team020)$ gcloud sql connect cs411-team020-mysql --user=team020user
Starting Cloud SQL Proxy: [/usr/bin/cloud-sql-proxy cs411-team020:us-central1:cs411-team020-mysql --port 9470]
2026/04/03 01:16:58 Authorizing with Application Default Credentials
2026/04/03 01:16:58 [cs411-team020:us-central1:cs411-team020-mysql] Listening on 127.0.0.1:9470
2026/04/03 01:16:58 The proxy has started successfully and is ready for new connections!
Connecting to database with SQL user [team020user].Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13659
Server version: 8.4.8-google (Google)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| nba_analytics |
| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)

mysql> USE nba_analytics;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_nba_analytics |
+-----+
| Arena |
| Game |
| GameOfficial |
| Official |
| Player |
| PlayerGameStats |
| Team |
| TeamGameStats |
+-----+
8 rows in set (0.01 sec)

mysql> SELECT DATABASE();
+-----+
| DATABASE() |
+-----+
| nba_analytics |
+-----+
1 row in set (0.01 sec)

mysql> █
```

1.1 MAIN TABLES

1 SELECT table_name 2 FROM information_schema.tables 3 WHERE table_schema = 'nba_analytics' 4 ORDER BY table_name;	Results	Execution time: 1.0 ms	Export	
TABLE_NAME				
Arena				
Game				
GameOfficial				
Official				
Player				
PlayerGameStats				
Team				
TeamGameStats				
Rows per page: 20	1 - 8 of 8			

1.2 CREATE TABLE DDL

The baseline results for Query 1, Query 2, and Query 3 were taken before we added the three test indexes for each query. Some tables already had indexes shown in the SHOW CREATE TABLE output, like primary key indexes, foreign key related indexes, or other indexes that were already part of the database. These were part of the original schema and were not the new indexes we were testing. In each query section, “Index Design A/B/C” only refers to the new test indexes added for that experiment.

Commands:

```
USE nba_analytics;
```

```
SHOW CREATE TABLE Arena;
```

```
SHOW CREATE TABLE Game;
```

```
SHOW CREATE TABLE Official;
```

```
SHOW CREATE TABLE GameOfficial;
```

```
SHOW CREATE TABLE Player;
```

```
SHOW CREATE TABLE PlayerGameStats;
```

```
SHOW CREATE TABLE Team;
```

```
SHOW CREATE TABLE TeamGameStats;
```

- Arena

Table	Create Table
Arena	CREATE TABLE `Arena` (`arenald` int NOT NULL, `name` varchar(150) NOT NULL, `city` varchar(100) DEFAULT NULL, `state` varchar(100) DEFAULT NULL, PRIMARY KEY (`arenald`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

- Game

Table	Create Table
Game	<pre>CREATE TABLE `Game` (`gameId` int NOT NULL, `gameDate` datetime NOT NULL, `arenaId` int NOT NULL, PRIMARY KEY (`gameId`), KEY `arenaId` (`arenaId`), CONSTRAINT `Game_ibfk_1` FOREIGN KEY (`arenaId`) REFERENCES `Arena` (`arenaId`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci</pre>

- GameOfficial

Table	Create Table
GameOfficial	<pre>CREATE TABLE `GameOfficial` (`gameId` int NOT NULL, `officialId` int NOT NULL, PRIMARY KEY (`gameId`,`officialId`), KEY `officialId` (`officialId`), CONSTRAINT `GameOfficial_ibfk_1` FOREIGN KEY (`gameId`) REFERENCES `Game` (`gameId`), CONSTRAINT `GameOfficial_ibfk_2` FOREIGN KEY (`officialId`) REFERENCES `Official` (`officialId`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci</pre>

- Official

Table	Create Table
Official	<pre>CREATE TABLE `Official` (`officialId` int NOT NULL, `fullName` varchar(150) NOT NULL, PRIMARY KEY (`officialId`), UNIQUE KEY `fullName` (`fullName`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci</pre>

- Player

Table	Create Table
Player	<pre>CREATE TABLE `Player` (`playerId` int NOT NULL, `firstName` varchar(100) NOT NULL, `lastName` varchar(100) NOT NULL, `heightInches` int DEFAULT NULL, `bodyWeightLbs` int DEFAULT NULL, PRIMARY KEY (`playerId`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci</pre>

- PlayerGameStats

Table	Create Table
PlayerGameStats	<pre>CREATE TABLE `PlayerGameStats` (`gameId` int NOT NULL, `playerId` int NOT NULL, `teamId` int NOT NULL, `points` int DEFAULT NULL, `minutes` decimal(6,2) DEFAULT NULL, `turnovers` int DEFAULT NULL, `assists` int DEFAULT NULL, `blocks` int DEFAULT NULL, `steals` int DEFAULT NULL, `rebounds` int DEFAULT NULL, `fg_percent` decimal(8,4) DEFAULT NULL, `three_percent` decimal(8,4) DEFAULT NULL, `ft_percent` decimal(8,4) DEFAULT NULL, PRIMARY KEY (`gameId`,`playerId`), KEY `index_pgs_playerId` (`playerId`), KEY `idx_pgs_teamId` (`teamId`), KEY `index_points_pgs` (`points`), KEY `index_teamId_pgs` (`teamId`), KEY `idx_q2_pgs_player_team` (`playerId`,`teamId`), KEY `idx_q2_pgs_points` (`points`), KEY `idx_q2_pgs_player_team_points` (`playerId`,`teamId`,`points`), CONSTRAINT `PlayerGameStats_ibfk_1` FOREIGN KEY (`gameId`) REFERENCES `Game` (`gameId`), CONSTRAINT `PlayerGameStats_ibfk_2` FOREIGN KEY (`playerId`) REFERENCES `Player` (`playerId`), CONSTRAINT `PlayerGameStats_ibfk_3` FOREIGN KEY (`teamId`) REFERENCES `Team` (`teamId`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci</pre>

- Team

Table	Create Table
Team	<pre>CREATE TABLE `Team` (`teamId` int NOT NULL, `name` varchar(100) NOT NULL, `city` varchar(100) NOT NULL, PRIMARY KEY (`teamId`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci</pre>

- TeamGameStats

Table	Create Table
TeamGameStats	<pre>CREATE TABLE `TeamGameStats` (`gameId` int NOT NULL, `teamId` int NOT NULL, `teamScore` int NOT NULL, `assists` int DEFAULT NULL, `blocks` int DEFAULT NULL, `steals` int DEFAULT NULL, `rebounds` int DEFAULT NULL, `turnovers` int DEFAULT NULL, `fg_percent` decimal(8,4) DEFAULT NULL, `three_percent` decimal(8,4) DEFAULT NULL, `ft_percent` decimal(8,4) DEFAULT NULL, `isHome` tinyint(1) NOT NULL, `win` tinyint(1) NOT NULL, PRIMARY KEY (`gameId`,`teamId`), KEY `idx_tgs_team_isHome` (`teamId`,`isHome`), KEY `idx_q3_tgs_teamId` (`teamId`), KEY `idx_q3_tgs_team_isHome` (`teamId`,`isHome`), KEY `idx_q3_tgs_team_isHome_score` (`teamId`,`isHome`,`teamScore`), CONSTRAINT `TeamGameStats_ibfk_1` FOREIGN KEY (`gameId`) REFERENCES `Game` (`gameId`), CONSTRAINT `TeamGameStats_ibfk_2` FOREIGN KEY (`teamId`) REFERENCES `Team` (`teamId`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci</pre>

2) Data Inserted (1000+ rows in 3 tables)

```
1 USE nba_analytics;
2
3 SELECT
4   (SELECT COUNT(*) FROM Arena)           AS ArenaRows,
5   (SELECT COUNT(*) FROM Game)            AS GameRows,
6   (SELECT COUNT(*) FROM Official)        AS OfficialRows,
7   (SELECT COUNT(*) FROM Player)          AS PlayerRows,
8   (SELECT COUNT(*) FROM Team)            AS TeamRows,
9   (SELECT COUNT(*) FROM PlayerGameStats) AS PlayerGameStatsRows,
10  (SELECT COUNT(*) FROM TeamGameStats)    AS TeamGameStatsRows,
11  (SELECT COUNT(*) FROM GameOfficial)     AS GameOfficialRows;
```

Results Execution time: 630.7 ms [Export](#) [Fullscreen](#) [Dropdown](#)

ArenaRows	GameRows	OfficialRows	PlayerRows	TeamRows	PlayerGameStatsRows	TeamGameStatsRows	GameOfficialRows
274	73092	82	6687	41	1431322	146170	3639

Rows per page: 20 1 - 1 of 1 |< < > >|

Row counts observed

- ArenaRows = 274
 - GameRows = 73,092
 - OfficialRows = 82
 - PlayerRows = 6,687
 - TeamRows = 41
 - PlayerGameStatsRows = 1,431,322
 - TeamGameStatsRows = 146,170
 - GameOfficialRows = 3,639
-

3) Advanced SQL Queries (3 total)

A) Query 1 — Player PPG leaderboard (min 50 games)

Query1 – Output (Top 15 rows)

Untitled query

Run Save Format Clear Valid

```
1 # Query output (top 15 rows)
2 USE nba_analytics;
3
4 SELECT
5     p.playerId,
6     p.firstName,
7     p.lastName,
8     COUNT(*) AS games_played,
9     ROUND(AVG(pgs.points), 2) AS avg_points
10 FROM PlayerGameStats pgs
11 JOIN Player p
12     ON p.playerId = pgs.playerId
13 GROUP BY p.playerId, p.firstName, p.lastName
14 HAVING COUNT(*) >= 50
15 ORDER BY avg_points DESC
16 LIMIT 15;
```

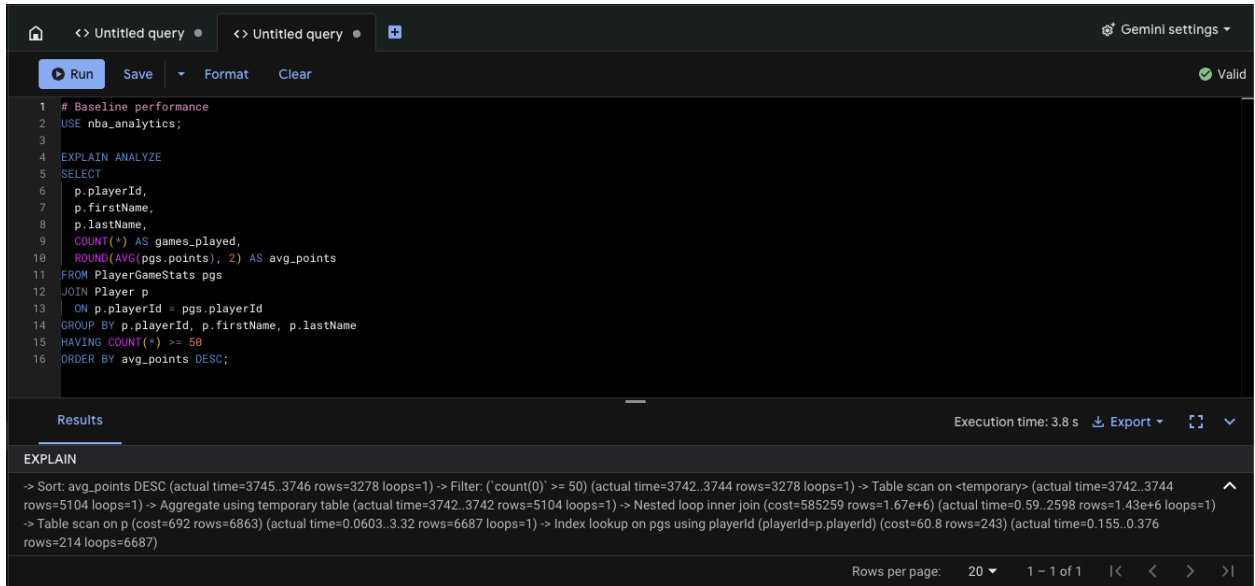
Results

Execution time: 3.6 s Export

playerId	firstName	lastName	games_played	avg_points
893	Michael	Jordan	1253	30.55
1629029	Luka	Doncic	607	28.15
78497	Jerry	West	1085	27.33
76127	Elgin	Baylor	818	27.29
201142	Kevin	Durant	1386	26.71
76804	George	Gervin	850	26.24
2544	LeBron	James	2068	25.64
78628	George	Yardley	125	25.00
252	Karl	Malone	1680	24.81
76003	Kareem	Abdul-Jabbar	1797	24.57
947	Allen	Iverson	1100	24.39
1628378	Donovan	Mitchell	714	24.38
76504	Adrian	Dantley	951	24.36
77264	Bernard	King	403	24.31
1628983	Shai	Gilgeous-Alexander	614	24.23

Rows per page: 20 1 - 15 of 15

Query1 – Baseline performance



The screenshot shows a SQL query editor with the following query:

```
1 # Baseline performance
2 USE nba_analytics;
3
4 EXPLAIN ANALYZE
5 SELECT
6   p.playerId,
7   p.firstName,
8   p.lastName,
9   COUNT(*) AS games_played,
10  ROUND(AVG(pgs.points), 2) AS avg_points
11 FROM PlayerGameStats pgs
12 JOIN Player p
13   ON p.playerId = pgs.playerId
14 GROUP BY p.playerId, p.firstName, p.lastName
15 HAVING COUNT(*) >= 50
16 ORDER BY avg_points DESC;
```

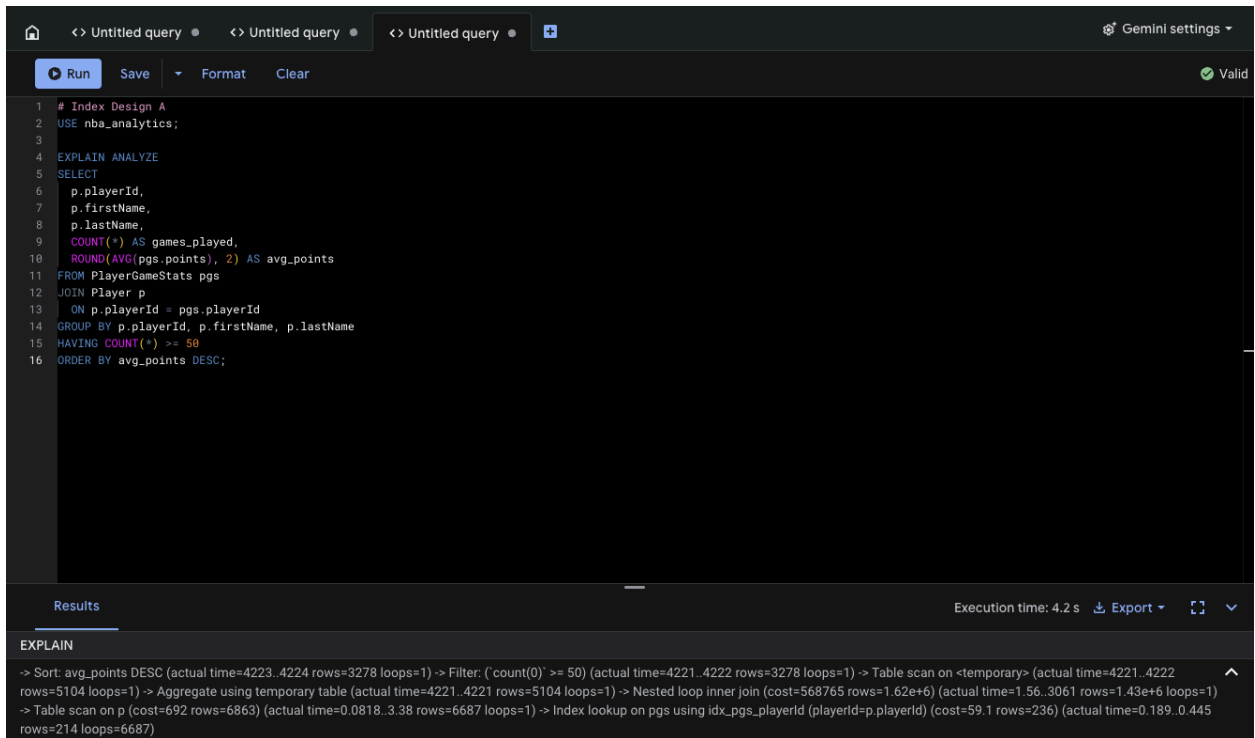
The query is executed, and the results are shown in the "Results" tab. The execution time is 3.8 s. The "EXPLAIN" tab shows the following execution plan:

EXPLAIN

-> Sort: avg_points DESC (actual time=3745.3746 rows=3278 loops=1) -> Filter: ('count(0)' >= 50) (actual time=3742.3744 rows=3278 loops=1) -> Table scan on <temporary> (actual time=3742.3744 rows=5104 loops=1) -> Aggregate using temporary table (actual time=3742.3742 rows=5104 loops=1) -> Nested loop inner join (cost=585259 rows=1.67e+6) (actual time=0.59.2598 rows=1.43e+6 loops=1) -> Table scan on p (cost=692 rows=6863) (actual time=0.0603.332 rows=6687 loops=1) -> Index lookup on pgs using playerId (playerId=p.playerId) (cost=60.8 rows=243) (actual time=0.155.0.376 rows=214 loops=6687)

Rows per page: 20 1 - 1 of 1

Query 1 - Index Design A



The screenshot shows the same SQL query as the baseline query, but with the "Index Design A" index applied. The query is:

```
1 # Index Design A
2 USE nba_analytics;
3
4 EXPLAIN ANALYZE
5 SELECT
6   p.playerId,
7   p.firstName,
8   p.lastName,
9   COUNT(*) AS games_played,
10  ROUND(AVG(pgs.points), 2) AS avg_points
11 FROM PlayerGameStats pgs
12 JOIN Player p
13   ON p.playerId = pgs.playerId
14 GROUP BY p.playerId, p.firstName, p.lastName
15 HAVING COUNT(*) >= 50
16 ORDER BY avg_points DESC;
```

The query is executed, and the results are shown in the "Results" tab. The execution time is 4.2 s. The "EXPLAIN" tab shows the following execution plan:

EXPLAIN

-> Sort: avg_points DESC (actual time=4223.4224 rows=3278 loops=1) -> Filter: ('count(0)' >= 50) (actual time=4221.4222 rows=3278 loops=1) -> Table scan on <temporary> (actual time=4221.4222 rows=5104 loops=1) -> Aggregate using temporary table (actual time=4221.4221 rows=5104 loops=1) -> Nested loop inner join (cost=568765 rows=1.62e+6) (actual time=1.56.3061 rows=1.43e+6 loops=1) -> Table scan on p (cost=692 rows=6863) (actual time=0.0818.338 rows=6687 loops=1) -> Index lookup on pgs using idx_pgs_playerId (playerId=p.playerId) (cost=59.1 rows=236) (actual time=0.189.0.445 rows=214 loops=6687)

Query 1 - Index Design B

Untitled query • <> Untitled query • <> Untitled query • +

Gemini settings ▾

Run Save Format Clear Valid

```
1 # Index Design B
2 USE nba_analytics;
3
4 CREATE INDEX idx_pgs_player_points ON PlayerGameStats(playerId, points);
5
6 EXPLAIN ANALYZE
7 SELECT
8   p.playerId,
9   p.firstName,
10  p.lastName,
11  COUNT(*) AS games_played,
12  ROUND(AVG(pgs.points), 2) AS avg_points
13 FROM PlayerGameStats pgs
14 JOIN Player p
15   ON p.playerId = pgs.playerId
16 GROUP BY p.playerId, p.firstName, p.lastName
17 HAVING COUNT(*) >= 50
18 ORDER BY avg_points DESC;
```

Results Execution time: 5.0 s Export ↕ ↕ Valid

EXPLAIN

-> Sort: avg_points DESC (actual time=1826..1827 rows=3278 loops=1) -> Filter: ('count(0)' >= 50) (actual time=1824..1825 rows=3278 loops=1) -> Table scan on <temporary> (actual time=1824..1825 rows=5104 loops=1) -> Aggregate using temporary table (actual time=1824..1824 rows=5104 loops=1) -> Nested loop inner join (cost=250763 rows=2.38e+6) (actual time=0.0801..691 rows=1.43e+6 loops=1) -> Table scan on p (cost=692 rows=6863) (actual time=0.0471..2.14 rows=6687 loops=1) -> Covering index lookup on pgs using idx_pgs_player_points (playerId=p.playerId) (cost=1.72 rows=347) (actual time=0.0363..0.0925 rows=214 loops=6687)

Rows per page: 20 ▾ 1 ~ 1 of 1 |< < > >|

Query 1 - Index Design C

Untitled query • <> Untitled query • <> Untitled query • +

Gemini settings ▾

Run Save Format Clear Valid

```
1 # Index Design C
2 USE nba_analytics;
3
4 CREATE INDEX idx_pgs_player_game_points ON PlayerGameStats(playerId, gameId, points);
5
6 EXPLAIN ANALYZE
7 SELECT
8   p.playerId,
9   p.firstName,
10  p.lastName,
11  COUNT(*) AS games_played,
12  ROUND(AVG(pgs.points), 2) AS avg_points
13 FROM PlayerGameStats pgs
14 JOIN Player p
15   ON p.playerId = pgs.playerId
16 GROUP BY p.playerId, p.firstName, p.lastName
17 HAVING COUNT(*) >= 50
18 ORDER BY avg_points DESC;
```

Results Execution time: 4.9 s Export ↕ ↕ Valid

EXPLAIN

-> Sort: avg_points DESC (actual time=1856..1856 rows=3278 loops=1) -> Filter: ('count(0)' >= 50) (actual time=1853..1855 rows=3278 loops=1) -> Table scan on <temporary> (actual time=1853..1855 rows=5104 loops=1) -> Aggregate using temporary table (actual time=1853..1853 rows=5104 loops=1) -> Nested loop inner join (cost=231201 rows=2.18e+6) (actual time=0.0916..709 rows=1.43e+6 loops=1) -> Table scan on p (cost=692 rows=6863) (actual time=0.0478..2.58 rows=6687 loops=1) -> Covering index lookup on pgs using idx_pgs_player_game_points (playerId=p.playerId) (cost=1.81 rows=318) (actual time=0.0376..0.094 rows=214 loops=6687)

Rows per page: 20 ▾ 1 ~ 1 of 1 |< < > >|

Query 1 conclusion:

This query finds the top scorers by average points per game for players who played at least 50 games. It joins Player and PlayerGameStats, groups by player, computes the average points, filters with HAVING COUNT(*) >= 50, and sorts by average points in descending order.

For the baseline query, the main plan cost was 585259.

For Index Design A, the cost became 568765.

For Index Design B, the cost became 250763.

For Index Design C, the cost became 231201.

This shows that Index Design A only gave a small improvement over the baseline. Index Design B gave a much larger improvement. Index Design C gave the lowest cost out of all four versions, so it was the best design for this query. This makes sense because the query groups by player and uses points for the average, so having an index that starts with playerId and also includes gameId and points helps the database read the needed rows more efficiently.

Because Index Design C had the lowest cost, we chose it as the best index for Query 1.

B) Query 2 — Players above league-wide average points

Query2 – Output (Top 15 rows)

```
1  -- Query2 - Output (Top 15 rows)
2  USE nba_analytics;
3
4  SELECT
5      p.playerId,
6      p.firstName,
7      p.lastName,
8      t.name AS teamName,
9      COUNT(pgs.gameId) AS gamesPlayed,
10     ROUND(AVG(pgs.points), 2) AS avgPoints,
11     ROUND(AVG(pgs.rebounds), 2) AS avgRebounds,
12     ROUND(AVG(pgs.assists), 2) AS avgAssists
13 FROM PlayerGameStats pgs
14 JOIN Player p ON pgs.playerId = p.playerId
15 JOIN Team t ON pgs.teamId = t.teamId
16 GROUP BY p.playerId, p.firstName, p.lastName, t.name
17 HAVING AVG(pgs.points) > (
18     SELECT AVG(points) FROM PlayerGameStats
19 )
20 ORDER BY avgPoints DESC
21 LIMIT 15;
```

Results

Execution time: 6.1 sExportFull ScreenDropdown

playerId	firstName	lastName	teamName	gamesPlayed	avgPoints	avgRebounds	avgAssists
893	Michael	Jordan	Bulls	1109	31.80	6.30	5.44
1629029	Luka	Doncic	Lakers	101	30.61	7.37	7.47
76003	Kareem	Abdul-Jabbar	Bucks	524	30.35	15.49	4.23
76504	Adrian	Dantley	Jazz	482	29.55	6.20	3.65
1122	Dominique	Wilkins	Clippers	25	29.08	7.04	2.20
201935	James	Harden	Rockets	772	28.02	5.70	7.32
1629029	Luka	Doncic	Mavericks	506	27.65	8.38	7.88
201142	Kevin	Durant	Thunder	697	27.53	7.28	3.75
76804	George	Gervin	Spurs	766	27.38	4.87	2.93
78497	Jerry	West	Lakers	1085	27.33	5.01	5.60
76127	Elgin	Baylor	Lakers	818	27.29	12.12	3.32
76375	Wilt	Chamberlain	76ers	321	27.25	24.42	6.75
406	Shaquille	O'Neal	Magic	331	26.98	12.39	2.56
77264	Bernard	King	Knicks	224	26.86	5.21	2.82
201142	Kevin	Durant	Nets	167	26.70	6.57	5.21

Rows per page: 201 – 15 of 15<<>>

Query2 – Baseline performance

```
1  -- Query2 - Baseline performance
2  USE nba_analytics;
3
4  EXPLAIN ANALYZE
5  SELECT
6    p.playerId,
7    p.firstName,
8    p.lastName,
9    t.name AS teamName,
10   COUNT(pgs.gameId) AS gamesPlayed,
11   ROUND(AVG(pgs.points), 2) AS avgPoints,
12   ROUND(AVG(pgs.rebounds), 2) AS avgRebounds,
13   ROUND(AVG(pgs.assists), 2) AS avgAssists
14 FROM PlayerGameStats pgs
15 JOIN Player p ON pgs.playerId = p.playerId
16 JOIN Team t ON pgs.teamId = t.teamId
17 GROUP BY p.playerId, p.firstName, p.lastName, t.name
18 HAVING AVG(pgs.points) > (
19   SELECT AVG(points) FROM PlayerGameStats
20 )
21 ORDER BY avgPoints DESC;
```

Results Execution time: 6.5 s Export

EXPLAIN

-> Sort: avgPoints DESC (actual time=6451.6451 rows=2383 loops=1) -> Filter: (??? > (select #2)) (actual time=6444.6449 rows=2383 loops=1) -> Table scan on <temporary> (actual time=6208.6211 rows=13222 loops=1) -> Aggregate using temporary table (actual time=6208.6208 rows=13222 loops=1) -> Nested loop inner join (cost=961021 rows=1.67e+6) (actual time=0.0716.4159 rows=1.43e+6 loops=1) -> Nested loop inner join (cost=376819 rows=1.67e+6) (actual time=0.0656.2251 rows=1.43e+6 loops=1) -> Table scan on t (cost=4.35 rows=41) (actual time=0.0192.0.0888 rows=41 loops=1) -> Index lookup on pgs using index_teamId_pgs (teamId=t.teamId) (cost=5219 rows=40711) (actual time=3.25.53.4 rows=34910 loops=41) -> Single-row index lookup on p using PRIMARY (playerId=pgs.playerId) (cost=0.25 rows=1) (actual time=0.00118..0.0012 rows=1 loops=1.43e+6) -> Select #2 (subquery in condition; run only once) -> Aggregate: avg(PlayerGameStats.points) (cost=286683 rows=1) (actual time=235.235 rows=1 loops=1) -> Covering index scan on PlayerGameStats using index_points_pgs (cost=144195 rows=1.42e+6) (actual time=0.429..154 rows=1.43e+6 loops=1)

Query2 – Index Design A

```
1  -- Query2 - Index Design A
2  USE nba_analytics;
3
4  CREATE INDEX idx_q2_pgs_player_team ON PlayerGameStats(playerId, teamId);
5
6  EXPLAIN ANALYZE
7  SELECT
8    p.playerId,
9    p.firstName,
10   p.lastName,
11   t.name AS teamName,
12   COUNT(pgs.gameId) AS gamesPlayed,
13   ROUND(AVG(pgs.points), 2) AS avgPoints,
14   ROUND(AVG(pgs.rebounds), 2) AS avgRebounds,
15   ROUND(AVG(pgs.assists), 2) AS avgAssists
16 FROM PlayerGameStats pgs
17 JOIN Player p ON pgs.playerId = p.playerId
18 JOIN Team t ON pgs.teamId = t.teamId
19 GROUP BY p.playerId, p.firstName, p.lastName, t.name
20 HAVING AVG(pgs.points) > (
21   SELECT AVG(points) FROM PlayerGameStats
22 )
```

Results Execution time: 9.7 s Export

EXPLAIN

-> Sort: avgPoints DESC (actual time=6439.6439 rows=2383 loops=1) -> Filter: (??? > (select #2)) (actual time=6432.6437 rows=2383 loops=1) -> Table scan on <temporary> (actual time=6196.6199 rows=13222 loops=1) -> Aggregate using temporary table (actual time=6196.6196 rows=13222 loops=1) -> Nested loop inner join (cost=961021 rows=1.67e+6) (actual time=0.0625.4153 rows=1.43e+6 loops=1) -> Nested loop inner join (cost=376819 rows=1.67e+6) (actual time=0.0494.2258 rows=1.43e+6 loops=1) -> Table scan on t (cost=4.35 rows=41) (actual time=0.00956..0.0883 rows=41 loops=1) -> Index lookup on pgs using index_teamId_pgs (teamId=t.teamId) (cost=5219 rows=40711) (actual time=3.24.53.5 rows=34910 loops=41) -> Single-row index lookup on p using PRIMARY (playerId=pgs.playerId) (cost=0.25 rows=1) (actual time=0.00118..0.0012 rows=1 loops=1.43e+6) -> Select #2 (subquery in condition; run only once) -> Aggregate: avg(PlayerGameStats.points) (cost=286683 rows=1) (actual time=235.235 rows=1 loops=1) -> Covering index scan on PlayerGameStats using index_points_pgs (cost=144195 rows=1.42e+6) (actual time=0.43..153 rows=1.43e+6 loops=1)

Query2 – Index Design B

```
1  -- Query2 - Index Design B
2  USE nba_analytics;
3
4  CREATE INDEX idx_q2_pgs_points ON PlayerGameStats(points);
5
6  EXPLAIN ANALYZE
7  SELECT
8      p.playerId,
9      p.firstName,
10     p.lastName,
11     t.name AS teamName,
12     COUNT(pgs.gameId) AS gamesPlayed,
13     ROUND(AVG(pgs.points), 2) AS avgPoints,
14     ROUND(AVG(pgs.rebounds), 2) AS avgRebounds,
15     ROUND(AVG(pgs.assists), 2) AS avgAssists
16 FROM PlayerGameStats pgs
17 JOIN Player p ON pgs.playerId = p.playerId
18 JOIN Team t ON pgs.teamId = t.teamId
19 GROUP BY p.playerId, p.firstName, p.lastName, t.name
20 HAVING AVG(pgs.points) > (
21     SELECT AVG(points) FROM PlayerGameStats
22 )
23 ORDER BY avgPoints DESC;
```

Results Execution time: 9.8 s Export

EXPLAIN

-> Sort: avgPoints DESC (actual time=6439.6439 rows=2383 loops=1) -> Filter: (??? > (select #2)) (actual time=6431.6437 rows=2383 loops=1) -> Table scan on <temporary> (actual time=6196.6200 rows=13222 loops=1) -> Aggregate using temporary table (actual time=6196.6196 rows=13222 loops=1) -> Nested loop inner join (cost=961021 rows=1.67e+6) (actual time=0.0509.4155 rows=1.43e+6 loops=1) -> Nested loop inner join (cost=376819 rows=1.67e+6) (actual time=0.0459.2258 rows=1.43e+6 loops=1) -> Table scan on t (cost=4.35 rows=41) (actual time=0.00939.0.0852 rows=41 loops=1) -> Index lookup on pgs using index_teamId_pgs (teamId=t.teamId) (cost=5219 rows=40711) (actual time=3.24.53.5 rows=34910 loops=41) -> Single-row index lookup on p using PRIMARY (playerId=pgs.playerId) (cost=0.25 rows=1) (actual time=0.00118.0.0012 rows=1 loops=1) -> Select #2 (subquery in condition; run only once) -> Aggregate: avg(PlayerGameStats.points) (cost=286683 rows=1) (actual time=235.235 rows=1 loops=1) -> Covering index scan on PlayerGameStats using index_points_pgs (cost=144195 rows=1.42e+6) (actual time=0.428.153 rows=1.43e+6 loops=1)

Query2 – Index Design C

```
1  -- Query2 - Index Design C
2  USE nba_analytics;
3
4  CREATE INDEX idx_q2_pgs_player_team_points ON PlayerGameStats(playerId, teamId, points);
5
6  EXPLAIN ANALYZE
7  SELECT
8      p.playerId,
9      p.firstName,
10     p.lastName,
11     t.name AS teamName,
12     COUNT(pgs.gameId) AS gamesPlayed,
13     ROUND(AVG(pgs.points), 2) AS avgPoints,
14     ROUND(AVG(pgs.rebounds), 2) AS avgRebounds,
15     ROUND(AVG(pgs.assists), 2) AS avgAssists
16 FROM PlayerGameStats pgs
17 JOIN Player p ON pgs.playerId = p.playerId
18 JOIN Team t ON pgs.teamId = t.teamId
19 GROUP BY p.playerId, p.firstName, p.lastName, t.name
20 HAVING AVG(pgs.points) > (
21     SELECT AVG(points) FROM PlayerGameStats
22 )
```

Results Execution time: 10.5 s Export

EXPLAIN

-> Filter: (??? > (select #2)) (actual time=6433.6438 rows=2383 loops=1) -> Table scan on <temporary> (actual time=6197.6200 rows=13222 loops=1) -> Aggregate using temporary table (actual time=6197.6197 rows=13222 loops=1) -> Nested loop inner join (cost=961021 rows=1.67e+6) (actual time=0.0535.4161 rows=1.43e+6 loops=1) -> Nested loop inner join (cost=376819 rows=1.67e+6) (actual time=0.0477.2270 rows=1.43e+6 loops=1) -> Table scan on t (cost=4.35 rows=41) (actual time=0.00966.0.0878 rows=41 loops=1) -> Index lookup on pgs using index_teamId_pgs (teamId=t.teamId) (cost=5219 rows=40711) (actual time=3.31.53.8 rows=34910 loops=41) -> Single-row index lookup on p using PRIMARY (playerId=pgs.playerId) (cost=0.25 rows=1) (actual time=0.00118.0.0012 rows=1 loops=1) -> Select #2 (subquery in condition; run only once) -> Aggregate: avg(PlayerGameStats.points) (cost=286683 rows=1) (actual time=236.236 rows=1 loops=1) -> Covering index scan on PlayerGameStats using index_points_pgs (cost=144195 rows=1.42e+6) (actual time=0.428.154 rows=1.43e+6 loops=1)

Query 2 conclusion:

Query 2 finds players whose average points are above the overall league average, while also showing their average rebounds and assists for each team they played on. It joins PlayerGameStats, Player, and Team, groups by player and team, uses a subquery in the HAVING clause, and sorts by average points in descending order.

For the baseline query, the main plan cost was 961021. For Index Design A, the cost was also 961021. For Index Design B, the cost was again 961021. For Index Design C, the cost was still 961021. This means none of the tested index designs improved the overall estimated cost of the query. The query still has to do a large amount of grouping, aggregation, and sorting, and it also includes a subquery in the HAVING condition, so those parts still dominate the plan. Because all four versions had the same cost, none of the index designs gave a clear benefit for Query 2.

No index design improved the overall cost

- Baseline: 961021
- A: 961021
- B: 961021
- C: 961021

C) Query 3 — Team Average Score Home Away

Query3 – Output (Top 15 rows)

<pre>1 -- Query3 - Output (Top 15 rows) 2 USE nba_analytics; 3 4 SELECT 5 t.teamId, 6 t.name AS teamName, 7 t.city, 8 ROUND(AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END), 2) AS avgHomeScore, 9 ROUND(AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END), 2) AS avgAwayScore, 10 ROUND(11 AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END) - 12 AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END), 13 2) AS homeAwayDiff 14 FROM TeamGameStats tgs 15 JOIN Team t 16 ON tgs.teamId = t.teamId 17 GROUP BY t.teamId, t.name, t.city 18 HAVING AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END) > 19 AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END) 20 ORDER BY homeAwayDiff DESC 21 LIMIT 15;</pre>					
Results					
Execution time: 375.5 ms Export Full Screen Dropdown					
teamId	teamName	city	avgHomeScore	avgAwayScore	homeAwayDiff
1610612743	Nuggets	Denver	110.88	105.46	5.42
1610612737	Hawks	Atlanta	105.99	101.13	4.85
1610612762	Jazz	Utah	106.06	101.45	4.61
1610612744	Warriors	Golden State	109.28	104.69	4.59
1610612757	Trail Blazers	Portland	107.32	103.00	4.32
1610612738	Celtics	Boston	108.41	104.18	4.23
1610612759	Spurs	San Antonio	107.28	103.04	4.23
1610612748	Heat	Miami	101.92	97.90	4.02
1610612747	Lakers	Los Angeles	108.78	104.81	3.96
1610612760	Thunder	Oklahoma City	107.99	104.04	3.95
1610612756	Suns	Phoenix	109.71	105.88	3.83
1610612755	76ers	Philadelphia	106.81	103.02	3.79
1610612764	Wizards	Washington	106.51	102.78	3.73
1610612739	Cavaliers	Cleveland	103.59	99.96	3.63
1610612749	Bucks	Milwaukee	107.02	103.46	3.56
Rows per page: 20 1 - 15 of 15 < < > >					

Query3 – Baseline performance

```
1 -- Query3 - Baseline performance
2 USE nba_analytics;
3
4 EXPLAIN ANALYZE
5 SELECT
6     t.teamId,
7     t.name AS teamName,
8     t.city,
9     AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END) AS avgHomeScore,
10    AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END) AS avgAwayScore,
11    (
12        AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END) -
13        AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END)
14    ) AS homeAwayDiff
15 FROM TeamGameStats tgs
16 JOIN Team t
17     ON tgs.teamId = t.teamId
18 GROUP BY t.teamId, t.name, t.city
19 HAVING AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END) >
20         AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END)
21 ORDER BY homeAwayDiff DESC
22 LIMIT 15;
```

Results Execution time: 388.7 ms [Export](#) [Full Screen](#) [Dropdown](#)

EXPLAIN

-> Limit: 15 row(s) (actual time=386.386 rows=15 loops=1) -> Sort: homeAwayDiff DESC (actual time=386.386 rows=15 loops=1) -> Filter: (??? > ???) (actual time=386.386 rows=30 loops=1) -> Table scan on <temporary> (actual time=386.386 rows=34 loops=1) -> Aggregate using temporary table (actual time=386.386 rows=34 loops=1) -> Nested loop inner join (cost=37774 rows=170749) (actual time=0.0558..205 rows=146170 loops=1) -> Table scan on t (cost=4.35 rows=41) (actual time=0.0272..0.0765 rows=41 loops=1) -> Index lookup on tgs using idx_tgs_team_isHome (teamId=t.teamId) (cost=515 rows=4165) (actual time=0.393..4.8 rows=3565 loops=41)

Rows per page: 20 1 - 1 of 1 |< < > >|

Query3 – Index Design A

```
1 -- Query3 - Index Design A
2 USE nba_analytics;
3
4 CREATE INDEX idx_q3_tgs_teamId ON TeamGameStats(teamId);
5
6 EXPLAIN ANALYZE
7 SELECT
8     t.teamId,
9     t.name AS teamName,
10    t.city,
11    AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END) AS avgHomeScore,
12    AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END) AS avgAwayScore,
13    (
14        AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END) -
15        AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END)
16    ) AS homeAwayDiff
17 FROM TeamGameStats tgs
18 JOIN Team t
19     ON tgs.teamId = t.teamId
20 GROUP BY t.teamId, t.name, t.city
21 HAVING AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END) >
22         AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END)
23 ORDER BY homeAwayDiff DESC
24 LIMIT 15;
```

Results Execution time: 690.3 ms [Export](#) [Full Screen](#) [Dropdown](#)

EXPLAIN

-> Limit: 15 row(s) (actual time=393.393 rows=15 loops=1) -> Sort: homeAwayDiff DESC (actual time=393.393 rows=15 loops=1) -> Filter: (??? > ???) (actual time=393.393 rows=30 loops=1) -> Table scan on <temporary> (actual time=393.393 rows=34 loops=1) -> Aggregate using temporary table (actual time=393.393 rows=34 loops=1) -> Nested loop inner join (cost=37774 rows=170749) (actual time=0.023..215 rows=146170 loops=1) -> Table scan on t (cost=4.35 rows=41) (actual time=0.00899..0.075 rows=41 loops=1) -> Index lookup on tgs using idx_tgs_team_isHome (teamId=t.teamId) (cost=515 rows=4165) (actual time=0.417..5.04 rows=3565 loops=41)

Rows per page: 20 1 - 1 of 1 |< < > >|

Query3 – Index Design B

```
1 -- Query3 - Index Design B
2 USE nba_analytics;
3
4 CREATE INDEX idx_q3_tgs_team_isHome ON TeamGameStats(teamId, isHome);
5
6 EXPLAIN ANALYZE
7 SELECT
8     t.teamId,
9     t.name AS teamName,
10    t.city,
11    AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END) AS avgHomeScore,
12    AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END) AS avgAwayScore,
13    (
14        AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END) -
15        AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END)
16    ) AS homeAwayDiff
17 FROM TeamGameStats tgs
18 JOIN Team t
19     ON tgs.teamId = t.teamId
20 GROUP BY t.teamId, t.name, t.city
21 HAVING AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END) >
22         AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END)
23 ORDER BY homeAwayDiff DESC
24 LIMIT 15;
```

Results Execution time: 709.4 ms [Export](#) [Full Screen](#) [Close](#)

EXPLAIN

-> Limit: 15 row(s) (actual time=389.389 rows=15 loops=1) -> Sort: homeAwayDiff DESC (actual time=389.389 rows=15 loops=1) -> Filter: (??? > ???) (actual time=389.389 rows=30 loops=1) -> Table scan on <temporary> (actual time=389.389 rows=34 loops=1) -> Aggregate using temporary table (actual time=389.389 rows=34 loops=1) -> Nested loop inner join (cost=37774 rows=170749) (actual time=0.0242..209 rows=146170 loops=1) -> Table scan on t (cost=4.35 rows=41) (actual time=0.0102..0.0682 rows=41 loops=1) -> Index lookup on tgs using idx_tgs_team_isHome (teamId=t.teamId) (cost=515 rows=4165) (actual time=0.398..4.9 rows=3565 loops=41)

Query3 – Index Design C

```
1 -- Query3 - Index Design C
2 USE nba_analytics;
3
4 CREATE INDEX idx_q3_tgs_team_isHome_score ON TeamGameStats(teamId, isHome, teamScore);
5
6 EXPLAIN ANALYZE
7 SELECT
8     t.teamId,
9     t.name AS teamName,
10    t.city,
11    AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END) AS avgHomeScore,
12    AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END) AS avgAwayScore,
13    (
14        AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END) -
15        AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END)
16    ) AS homeAwayDiff
17 FROM TeamGameStats tgs
18 JOIN Team t
19     ON tgs.teamId = t.teamId
20 GROUP BY t.teamId, t.name, t.city
21 HAVING AVG(CASE WHEN tgs.isHome = 1 THEN tgs.teamScore END) >
22         AVG(CASE WHEN tgs.isHome = 0 THEN tgs.teamScore END)
23 ORDER BY homeAwayDiff DESC
24 LIMIT 15;
```

Results Execution time: 646.4 ms [Export](#) [Full Screen](#) [Close](#)

EXPLAIN

-> Limit: 15 row(s) (actual time=245.245 rows=15 loops=1) -> Sort: homeAwayDiff DESC (actual time=245.245 rows=15 loops=1) -> Filter: (??? > ???) (actual time=245.245 rows=30 loops=1) -> Table scan on <temporary> (actual time=245.245 rows=34 loops=1) -> Aggregate using temporary table (actual time=245.245 rows=34 loops=1) -> Nested loop inner join (cost=17474 rows=170749) (actual time=0.0223..67.3 rows=146170 loops=1) -> Table scan on t (cost=4.35 rows=41) (actual time=0.0108..0.0761 rows=41 loops=1) -> Covering index lookup on tgs using idx_q3_tgs_team_isHome_score (teamId=t.teamId) (cost=19.8 rows=4165) (actual time=0.132..1.45 rows=3565 loops=41)

Query 3 conclusion:

Query 3 compares each team's average home score and average away score, and then ranks teams by the difference between the two. It joins TeamGameStats and Team, groups by team, computes conditional averages using CASE WHEN, filters teams whose average home score is greater than their average away score, and sorts by the home-away difference in descending order.

For the baseline query, the main cost was 37774. Index Design A and Index Design B both resulted in the same cost of 37774, which means there was no improvement. This might be because indexes do not cover all attributes used in the query's aggregation, which means MySQL would still have to access the table for teamScore. Additionally, Index Design C had a cost of 17474, which was an improvement. This improvement probably occurs because Index C acts as a covering index, which includes teamId, isHome, and teamScore, which are all used for the join, grouping, and aggregation. This means MySQL can perform a covering index and avoid table lookups, which takes more time. This means Index C is the best index for query 3.

- Baseline: 37774
- A: 37774
- B: 37774
- C: 17474

Index C design improved the overall cost

Query Conclusions:

- **Query 1:** Index C is best
- **Query 2:** None of the tested indexes improved the cost
- **Query 3:** Index C is best